

## Los castores en el límite de la computación

por

Sergio Miguéns Iglesias

La frontera de la computación, aquella que separa los algoritmos que terminan de los que no, los problemas computables y los no computables, no es un concepto abstracto o difuso, sino que puede ser explorada analizando el comportamiento de la función *busy beaver*  $BB(n)$  («castor hacendoso u ocupado» en español). Esta función, entre otras curiosidades, tiene la propiedad de crecer más rápido que cualquier función calculable que podamos imaginar, como  $n^2$ ,  $2^n$ ,  $10^{n!}$  o la función  $TREE(n)$ <sup>1</sup>, famosa por su rápido crecimiento. De hecho, podemos interpretar que  $BB(n)$  crece tan rápido que, para  $n \geq 745$ ,  $BB(n)$  se hace independiente de los axiomas de Zermelo–Fraenkel, escapando así de nuestra teoría de conjuntos estándar. Conocer  $BB(n)$  para valores elevados de  $n$  resuelve también problemas abiertos sin relación aparente, como la conjetura de Collatz o la hipótesis de Riemann, exploradas más adelante.

El juego del castor hacendoso nace del problema de la parada de Alan Turing, según el cual no existe ningún algoritmo capaz de predecir si cualquier algoritmo, dados unos datos de entrada, termina o no.

Por facilidad de análisis, representamos los algoritmos como máquinas de Turing: máquinas dotadas de una cinta de datos de la que pueden leer y escribir, y de un conjunto de estados que les indican cómo actuar en cada paso. Para un número  $n$  de estados dado, la *busy beaver* de  $n$  estados es aquella máquina/algoritmo que, dada una cinta llena de ceros, se detiene tras dar la mayor cantidad de pasos posible.  $BB(n)$  es entonces la cantidad de pasos que esa máquina ha dado, la cota superior de pasos que cualquier máquina con  $n$  estados puede dar antes de parar, representando la linde de lo computable.

Desgraciadamente, la naturaleza de  $BB(n)$  es tal que descubrir sus valores más allá del trivial  $BB(1) = 1$  se complica tanto como crece la propia función. Pese a ello, matemáticos profesionales y aficionados han buscado sus valores por los últimos sesenta años, logrando confirmar el valor de  $BB(5)$  tras un monumental esfuerzo comunitario. Esta búsqueda ha sido llamada «el juego de los castores hacendosos», *the busy beaver game*, al que dedicaremos estas páginas para explicarlo en profundidad y explorar sus seis décadas de historia.

---

<sup>1</sup>Una función que cuenta la longitud de una secuencia de árboles con nodos etiquetados con  $n$  símbolos. Cada árbol de la secuencia ha de ser «único», es decir, no contener como subestructura a ninguno de los árboles que le preceden en la misma [8]. Sus primeros valores son los siguientes:  $TREE(1) = 1$ ,  $TREE(2) = 3$  y  $TREE(3) \gg 10^{10^{100}}$ .

## 1. LAS MÁQUINAS DE TURING

Una máquina de Turing  $M$  es una abstracción matemática muy útil para representar algoritmos, desarrollada el siglo pasado por Alan Turing. La máquina  $M$  lee datos de una cinta infinita en ambas direcciones dividida en celdas, y escribe nuevos datos a medida que se desplaza por ella. En cada paso, la máquina de Turing lee un dato de la cinta, lo actualiza (o no) en esa posición, y se mueve a una celda contigua, donde repite el proceso.

La máquina sabe qué hacer en cada situación porque está dotada de estados y de una función de transición que le indica, según el dato leído y su estado actual: (1) qué escribir en la cinta, (2) en qué dirección moverse, y (3) a qué nuevo estado cambiar. Entre ellos, el inicial es aquel en el que comienza a operar, y los estados finales son los que hacen que la máquina se detenga tras alcanzarlos, momento en el que decimos que ha «terminado» o «parado». Por ejemplo, en la figura 1, 0: 1R significa que tras leer 0 en la cinta: (1) se sobrescriba con 1, (2) se realice un desplazamiento a la derecha (*Right*) y (3) se cambie al estado apuntado por la flecha.

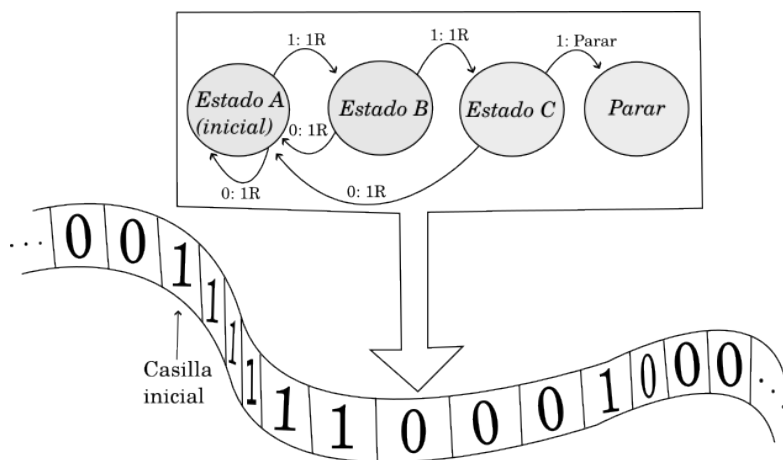


Figura 1: Máquina de Turing de tres estados que busca tríos de 1s en la cinta a la derecha de su posición inicial, dejando 1s a su paso.

En resumen, una máquina de Turing  $M$  está dotada de una cinta infinita por la que se va desplazando, guiada por sus estados, mientras lee y escribe información sobre la misma, y que termina, o no, según lo que indiquen sus estados, como se ve en la figura 1.

Los datos en la cinta son siempre elementos del conjunto de símbolos de la cinta  $\Gamma$ , pudiendo ser este cualquier conjunto finito. Por simplicidad, y sin pérdida de generalidad, nos centraremos en máquinas de Turing binarias, aquellas para las cuales  $\Gamma = \{0, 1\}$ <sup>2</sup>. Al conjunto de estas máquinas lo llamaremos  $\mathcal{M}^2$ , y sus subconjuntos

<sup>2</sup>O  $|\Gamma| = 2$ , ya que el nombre particular de los símbolos es irrelevante en este caso, lo importante es la cardinalidad del conjunto y que con dos símbolos basta para representar información, como en nuestros ordenadores convencionales.

$\mathcal{M}_n^2 \subset \mathcal{M}^2$  contendrán aquellas máquinas de Turing binarias con  $n$  estados. Por supuesto, **cuantos más estados tiene la máquina, más complejas son las operaciones que puede realizar**. Una máquina que cuente hasta cinco necesita bastantes menos estados que una que, por ejemplo, ejecute un modelo de lenguaje moderno. La complejidad del algoritmo modelado y la cantidad mínima de estados requeridos en la máquina que lo modela van siempre de la mano.

| Estado actual | Dato en cinta | Acción |
|---------------|---------------|--------|
| A (inicial)   | 0             | ORA    |
| A (inicial)   | 1             | 1RB    |
| B             | 0             | ORA    |
| B             | 1             | 1RC    |
| C             | 0             | ORA    |
| C             | 1             | Parar  |

Tabla 1: tabla de transiciones de la máquina de Turing de la figura 1.

Veamos un caso práctico con la tabla 1. A ese tipo de tablas las llamamos «tablas de transición», y se utilizan para describir y visualizar la función de transición de una máquina de Turing, en este caso la de la figura 1. Por ejemplo, suponiendo que la máquina de la tabla 1 está en el estado B y que en la cinta ha leído un 0, su acción será ORA, es decir, mantener el 0 que había en la cinta, moverse a la derecha en esta (*Right*) y cambiar al estado A.

Si estudiamos esa máquina en profundidad, veremos que se desplaza hacia la derecha ignorando los ceros que encuentra. Solo cambia del estado A al B si encuentra un 1, y lo mismo del B al C, donde solo se detiene al encontrar un tercer 1. Si encuentra un 0 antes de parar, comienza de nuevo el proceso en A. En definitiva, la máquina solo se detiene sobre la cinta si y solo si en esta hay tres o más 1s consecutivos, ignorando todo 0 y grupos menores de tres 1s que encuentre en el camino.

## 2. EL PROBLEMA DE LA PARADA

El problema de la parada, formulado también por Alan Turing, nace de una pregunta natural sobre estas máquinas: ¿Podemos saber con antelación si una máquina se detendrá al ejecutarse sobre una cinta dada? [26]. La conclusión de Turing fue clara: «no existe ninguna máquina de Turing  $H$  que pueda decidir, para toda máquina de Turing  $M$  y cualquier cinta de entrada dada, si  $M$  parará o no en tiempo finito». Esta imposibilidad se demuestra mediante una reducción al absurdo: si tal máquina  $H$  existiera, sería posible construir a partir de ella una máquina accesoria  $X$  diseñada para contradecir el diagnóstico de  $H$ . Al intentar evaluar qué ocurriría al aplicar  $X$  a su propia descripción, surge una contradicción lógica irresoluble, donde  $X$  se detiene si y solo si  $H$  predice que no lo hará, invalidando la existencia inicial de  $H$ .

No se debe pensar que esto quiere decir que es imposible saber si una máquina de Turing concreta, o un conjunto de máquinas con cintas establecidas, se detendrán

| Estado actual | Dato en cinta | Acción |
|---------------|---------------|--------|
| A (inicial)   | 0             | 0RA    |
| A (inicial)   | 1             | 1RA    |
| B             | 0             | 1LA    |
| B             | 1             | Parar  |

Tabla 2: tabla de transiciones de una máquina de Turing de dos estados que nunca termina. Sea cual sea la cinta, la deja intacta y se mueve a la derecha indefinidamente.

o no, pues los análisis individualizados sí son posibles, como puede observarse en las tablas 2 y 3. De lo que nunca dispondremos es de un algoritmo que resuelva el problema para todas las máquinas imaginables con cintas arbitrarias.

| Estado actual | Dato en cinta | Acción |
|---------------|---------------|--------|
| A (inicial)   | 0             | 1LA    |
| A (inicial)   | 1             | Parar  |

Tabla 3: tabla de transiciones de una máquina de Turing de un estado que termina cuando encuentra un uno en la cinta, dejando a su paso una secuencia de 1s.

Dadas las descripciones de las tablas 2 y 3, es fácil ver que la primera máquina no termina jamás, mientras que la segunda termina si y solo si hay algún 1 a la izquierda de la posición inicial en la cinta. Es por esto que conviene recordar que el problema de la parada es una limitación metodológica general y no específica para cada máquina analizada.

Pasemos a continuación con un ejemplo más pertinente para este artículo, dado en la tabla 4:

| Estado actual | Dato en cinta | Acción |
|---------------|---------------|--------|
| A (inicial)   | 0             | 1RB    |
| A (inicial)   | 1             | 1LB    |
| B             | 0             | 1LA    |
| B             | 1             | Parar  |

Tabla 4: tabla de transiciones de la máquina que más pasos da en  $\mathcal{M}_2^2$  sobre una cinta de 0s.

En la tabla 4 se muestran los estados y transiciones de la máquina de Turing en  $\mathcal{M}_2^2$  que más pasos realiza sobre una cinta de 0s antes de terminar, seis en total. Esto la convierte, como veremos en la siguiente sección, en la segunda *busy beaver*, dándole valor a  $BB(2)$ . Sus cambios paso a paso sobre la cinta pueden verse en la tabla 5, donde el paréntesis indica el elemento que está siendo apuntado en cada momento. A este tipo de tablas se las llama tablas espaciotemporales, por mostrar cómo los elementos de la cinta (espacio) cambian en cada paso (tiempo). Estas

| Paso | Estado | Cinta |     |     |     |   | Instrucción |
|------|--------|-------|-----|-----|-----|---|-------------|
| 1    | A      | 0     | 0   | (0) | 0   | 0 | 1RB         |
| 2    | B      | 0     | 0   | 1   | (0) | 0 | 1LA         |
| 3    | A      | 0     | 0   | (1) | 1   | 0 | 1LB         |
| 4    | B      | 0     | (0) | 1   | 1   | 0 | 1LA         |
| 5    | A      | (0)   | 1   | 1   | 1   | 0 | 1RB         |
| 6    | B      | 1     | (1) | 1   | 1   | 0 | Parar       |

Tabla 5: tabla de «espaciotiempo» de la máquina descrita en la tabla 4.

tablas, y su evolución natural, los diagramas espaciotemporales<sup>3</sup>, son herramientas visuales comunes con las que los participantes en el juego intuyen el potencial de una máquina para detenerse tras dar una gran cantidad de pasos.

Ahora que conocemos a nuestro segundo castor, es hora de definir formalmente qué es la función  $BB(n)$ .

### 3. LA FUNCIÓN $BB(n)$

Antes de pasar a los castores que nos atañen, definamos un poco la notación requerida. Primero,  $\mathcal{M}^2$  será el conjunto de todas las máquinas de Turing con alfabeto binario, y  $\mathcal{M}_n^2 \subset \mathcal{M}^2$  el subconjunto de todas las máquinas de Turing de  $n$  estados. Segundo, la función  $|\cdot|_0 : \mathcal{M} \rightarrow \mathbb{Z}_{\geq 0} \cup \{\infty\}$  cuenta los pasos que una máquina de Turing realiza antes de parar dada una cinta con solo ceros. El valor de  $|M|_0$  es un número entero no negativo, el número de pasos previos a la detención de  $M$ ; o infinito, en caso de que la ejecución no termine nunca. De esta forma,  $|M|_0 = 15, M \in \mathcal{M}_5^2$  indica que la máquina de Turing binaria  $M$  tiene cinco estados, y que realiza 15 pasos antes de detenerse sobre una cinta de ceros.

Con esta notación, podemos definir una *busy beaver* como una máquina de Turing binaria  $M$  de  $n$  estados que, **dada una cinta de solo ceros, termina en un número de pasos finito igual o mayor que cualquier otra máquina de Turing en  $\mathcal{M}_n^2$  que también se detenga**. La cantidad de pasos que da esta máquina viene dada por la función:

$$BB(n) = \max_{M \in \mathcal{M}_n^2, |M|_0 \neq \infty} |M|_0.$$

Es decir, que para un número de estados dado, la *busy beaver* es aquella máquina que más pasos da sobre la cinta de ceros, pero siempre parando, lo que descarta todas aquellas máquinas cuya computación nunca termine.

Es por ese motivo que  $BB(n)$  crece más rápido que cualquier función computable, porque representa, por definición, la barrera de crecimiento para estas funciones. Supongamos por reducción al absurdo que existe una función computable

<sup>3</sup>No entraremos en detalle por no ser estos demasiado relevantes para el artículo, pero digamos que son imágenes de mapa de bits en blanco y negro, donde cada píxel blanco representa un 1 en la cinta, y cada píxel negro un 0. Cada fila de píxeles es un paso consecutivo de ejecución, y se omite la información sobre el estado actual de la máquina que fue incluida en la tabla. Según la forma dibujada, se puede intuir la evolución de la cinta, lo que ayuda a seleccionar máquinas interesantes.

$f(n)$  que cumple que  $f(n) \geq BB(n)$ . Como  $f(n)$  es una cota superior de  $BB(n)$ , si queremos saber si una máquina de  $m$  estados se detendrá sobre una cinta de ceros, solo necesitamos ejecutarla durante  $f(m)$  pasos. Tendríamos, de facto, una función que resuelve el problema de la parada, lo que es imposible [26]. Por lo tanto, ninguna función computable puede acotar superiormente a  $BB(n)$  y, en particular, ninguna puede crecer más rápido que ella.

Por este mismo razonamiento es interesante estudiar la función *busy beaver*: si conocemos el valor de  $BB(n)$ , y queremos saber si un algoritmo dado, codificable en una máquina de Turing  $M \in \mathcal{M}_n^2$ , termina o no, lo único que tenemos que hacer es ejecutar dicho algoritmo durante  $BB(n)$  pasos. Si llegamos al paso  $BB(n) + 1$ , sabemos que el algoritmo no va a terminar, pues  $BB(n)$  representa la mayor cantidad de pasos que puede dar  $M$  antes de detenerse. De esta forma, conocer  $BB(n)$  determina la parada de todas las máquinas  $M \in \mathcal{M}_n^2$ , lo que para ciertos valores de  $n$  permite encontrar la solución a varios problemas interesantes [1]:

- Si la conjetura de Goldbach<sup>4</sup> es falsa, existe una máquina de Turing de 27 estados que se detiene. Conocer  $BB(27)$ , por lo tanto, resuelve la conjetura de Goldbach.
- Igualmente, si la hipótesis de Riemann<sup>5</sup> es falsa, existe una máquina de Turing de 744 estados que se detiene, por lo que conocer  $BB(744)$  soluciona la hipótesis de Riemann.
- Lo mismo sucede con la conjetura de Paul Erdős<sup>6</sup>, siendo esta falsa si cierta máquina en  $\mathcal{M}_{15}^2$  termina. Conocer  $BB(15)$ , por lo tanto, resuelve la conjetura de Paul Erdős.

No sería incorrecto pensar que resolver estos problemas descubriendo primero el valor de  $BB(n)$  correspondiente es absurdo, porque  $BB(n)$  siempre es tan difícil de resolver, por lo menos, como cada problema concreto en  $\mathcal{M}_n^2$ . Al fin y al cabo, para conocer  $BB(n)$  se debe determinar previamente la parada o no de todas las máquinas en el conjunto  $\mathcal{M}_n^2$ , incluyendo la del problema en cuestión que queremos resolver. Sin embargo, y aparte de ser un buen reto intelectual, estudiar  $BB(n)$  para valores pequeños podría darnos buenas pistas sobre los métodos necesarios para abordar problemas similares, especialmente con la conjetura de Collatz<sup>7</sup>, cuya estructura se reproduce ya para  $n < 25$  (veáanse [15–17]).

Pero conocer valores altos de  $BB(n)$  podría complicarse, pues se ha demostrado que existe una máquina de Turing de 745 estados que se detiene solo si los axiomas

<sup>4</sup>Todo número entero mayor que 5 se puede escribir como suma de tres números primos [10].

<sup>5</sup>Esta establece que los ceros no triviales de cierta función compleja, llamada la función zeta de Riemann  $\zeta(s)$ , se encuentran siempre en la recta  $1/2 + iy$ ,  $y \in \mathbb{R}$  [24]. Si esto fuese cierto, los números primos estarían distribuidos de forma relativamente predecible, lo que tendría importantes consecuencias en campos como la teoría de números y la criptografía.

<sup>6</sup>La conjetura dice así: para todo  $n \geq 8$  siempre hay un 2 en la representación de  $2^n$  en base 3 [7].

<sup>7</sup>La conjetura de Collatz establece que, para cualquier  $n \in \mathbb{N}$ , si le aplicamos iterativamente la función:

$$f(n) = \begin{cases} n/2 & \text{si } n \text{ es par.} \\ 3n + 1 & \text{si } n \text{ es impar.} \end{cases} \quad (1)$$

Siempre se llega a 1, independientemente del  $n$  elegido.

de Zermelo-Fraenkel (ZFC) son inconsistentes [2, 22, 23, 28]. Esto quiere decir, por el segundo teorema de incompletitud de Gödel [11], que **es imposible conocer  $BB(745)$  usando la teoría de conjuntos estándar**, porque hacerlo requiere demostrar la inconsistencia de ZFC desde dentro del sistema. Decimos entonces que  $BB(745)$ , y  $BB(n)$  para todo  $n \geq 745$  [1], son independientes de los Axiomas de Zermelo-Fraenkel.

Peor aún es que Scott Aaronson, uno de los padres de las anteriores demostraciones, conjetura que podría existir una máquina de Turing similar de apenas 20 estados [1]. Si esto fuese cierto, significaría que no podemos conocer el valor de  $BB(n)$  para  $n \geq 20$  dentro de la teoría de conjuntos estándar.

#### 4. BUSCANDO CASTORES

Ahora que conocemos la función  $BB(n)$ , el siguiente paso lógico es descubrir sus valores, que, para máquinas de uno y dos estados ( $BB(1)$  y  $BB(2)$ ) fueron dados por Tibor Radó<sup>8</sup> en el año 1962 [21], en el mismo artículo en el que acuña el término *busy beaver*<sup>9</sup> y da comienzo al juego:

$$BB(1) = 1$$

$$BB(2) = 6$$

Radó volvió a dar un nuevo valor para  $BB(n)$  en 1965 junto a Shen Lin [19], que llevaba estudiando el problema dos años por aquel entonces [18]. El juego se complicaba, porque si bien  $BB(1) = 1$  era trivial ya que con un único estado solo puedes dar un paso y parar, o entrar en un bucle infinito; y  $BB(2) = 6$  se podía deducir por fuerza bruta;  $BB(3)$  suponía un salto de dificultad considerable, incluso siendo solo la tercera, necesitando un proceso relativamente complejo:

1. Redujeron el espacio de búsqueda en un factor de  $\approx 363$ , eliminando máquinas con comportamientos similares.
2. Conjeturaron que  $BB(3) = 21$ , así que toda máquina que parase antes fue descartada.
3. Las máquinas restantes fueron ejecutadas y monitorizadas. Si su evolución mostraba patrones repetitivos, se descartaban.
4. Verificaron manualmente que las máquinas restantes no paraban.

---

<sup>8</sup>Radó llamó  $S(n)$  a la función que ahora conocemos como  $BB(n)$ , y así aparece aún en muchas fuentes. La idea era que esta función contaba los pasos que la máquina de Turing daba (*shifts*) antes de parar, y sus valores eran solo objetivos secundarios del juego. Lo principal era encontrar los valores de  $\Sigma(n)$ , que cuenta (suma, de ahí su nombre, *sigma mayúscula*) los unos que la máquina ha dejado en la cinta. Los valores de esta función son siempre menores que los de  $BB(n)$ , y su relación con problemas abiertos menos intuitiva, hecho por el cual ha sido desplazada como objetivo principal del juego por  $BB(n)$ .

<sup>9</sup>Expresión anglosajona para referirse a una persona muy ocupada, adoptada en este caso por ser estas máquinas las que más pasos dan en sus respectivas familias.

Tras ese trabajo, se obtuvo que, tal y como se había conjeturado:

$$BB(3) = 21.$$

Después de eso, habría que esperar hasta abril de 1983 para que se publicase en *Mathematics of Computation* un artículo de Allen H. Brady con el valor de  $BB(4)$  [5], logro que Radó había considerado imposible exactamente veinte años antes. Para obtenerlo, tuvo que separar las máquinas cuya detención estaba aún indecisa en dos clases, contadores y árboles de Navidad, terminando con la conclusión:

$$BB(4) = 107.$$

Y ese sería el último descubrimiento en mucho tiempo, pues el incremento de dificultad que supone determinar  $BB(5)$  es tal que muchos pensaban que nunca llegaría el día en el que su valor fuese revelado.

## 5. APARECEN LOS AFICIONADOS

En el año 1990 dos autores, Marxen y Buntrock, conjeturaron que  $BB(5) = 47176870$  [20] tras reducir el espacio de búsqueda de millones a cientos de miles de máquinas. Pero el gran salto fue dado en el año 2003, cuando un matemático húngaro llamado Georgi Georgiev (apodado «*Skelet*») lo redujo aún más, dejando tan solo 43 candidatas [13]. Su trabajo sentó la base para futuros análisis, aunque la falta de revisión por pares del mismo hacía aconsejable cierta cautela. Esas 43 máquinas pasaron a llamarse las *Skelets*, las más difíciles de analizar en  $\mathcal{M}_5^2$ .

Para el año 2009 había ya otros autores con colecciones propias de máquinas extrañas en  $\mathcal{M}_5^2$ . Entre ellos estaban Joachim Hertel, que redujo de forma independiente la búsqueda a una lista de 100 máquinas [12], desconociendo el trabajo que *Skelet* había realizado años atrás; y Daniel Briggs, un alumno del MIT que determinó la parada de la mayoría de las *Skelet* en 2010 [6]. Su trabajo pasó desapercibido por décadas, porque sus inaccesibles demostraciones, además de informales, estaban alojadas en un repositorio de *GitHub*. Solo Scott Aaronson parecía ser consciente de su trabajo en 2016 [4].

El propio Aaronson, interesado ya por el juego, descubrió su primera máquina cuya parada no puede ser demostrada dentro de ZFC [28] y, en su artículo de 2020 [1], conjeturó que  $BB(5) = 47176870$ , coincidiendo con la conjetura original de Marxen y Buntrock [20] treinta años más tarde.

La mayor revolución llegó en el año 2022, con la formación de *BBChallenge* (<https://bbchallenge.org>), una comunidad dedicada a la búsqueda del elusivo quinto castor. Armados con razonadores inversos y herramientas de automatización de demostraciones, descartaron una a una las máquinas de *Skelet*, hasta que solo quedasen la primera, la décima y la decimoséptima, cada una requiriendo análisis muy distintos:

- La *Skelet 1* alterna entre comportamientos ordenados y caóticos por más de un millardo de pasos, pero se logró demostrar que después entra en un bucle infinito de  $8 \cdot 10^9$  pasos por iteración [13].



- La *Skelet 10*, por su lado, es un contador binario doble con codificación de Fibonacci [14]. Es decir, un contador que cuenta tanto hacia la derecha como hacia la izquierda desde el punto de partida de la cinta. Demostrar que nunca termina requirió probar que ambos contadores están sincronizados y son interdependientes, por lo que cuentan hasta el infinito.
- Por último, la *Skelet 17* contiene cuatro tipos distintos de patrones, que requirieron estudios individualizados para demostrar que nunca para [27].

En el año 2025, con las máquinas restantes resueltas, solo restaba formalizar la demostración, lo que se hizo en el asistente de demostración de teoremas *Rocq* [25]. El resultado fue que el valor de  $BB(5)$  es tal cual se había conjeturado hace casi medio siglo:

$$BB(5) = 47176870.$$

En tres años, una comunidad de aficionados logró lo que profesionales del sector creían casi imposible, demostrando la capacidad de las comunidades en línea para realizar avances matemáticos. Pero su búsqueda aún no ha terminado, pues nuevos retos aguardan a aquellos que se aventuren a continuar con el juego de Radó.

## 6. NUEVAS FRONTERAS

Con la búsqueda de  $BB(5)$  terminada, la comunidad de *BBChallenge* ha comenzado a buscar nuevos horizontes que explorar, como valores aún mayores de  $BB(n)$  u otros problemas relacionados.

El sucesor más natural para la búsqueda es, en este caso,  $BB(6)$ , para el que ya contamos con una cota inferior gracias a *Mxdys*, un aficionado de nombre real desconocido [3]:

$$BB(6) \geq {}^{9_2}_2 10.$$

En este caso, los exponentes a la derecha denotan una operación de tetración, o exponenciación repetida ( ${}^3 2 = 2^{2^2} = 16$ ), por lo que la cota mínima de  $BB(6)$  es tal que, si cada átomo del universo tuviese dentro un universo propio, y esta recursión se diese por mil iteraciones, no estaríamos ni cerca de tener tantos átomos en total como dígitos tiene este número.

Aaronson bromea con ese hecho [3], al mismo tiempo que actualiza su conjetura inicial de que  $BB(n), n \geq 20$  es independiente de ZFC para aventurarse a decir que es muy probable, *en su opinión*, que valores tan bajos como  $BB(8)$ ,  $BB(9)$  o  $BB(10)$  sean ya independientes de los axiomas de nuestra teoría estándar de conjuntos.

También se está poniendo el foco en extensiones de la función  $BB(n)$ , como por ejemplo encontrar las *busy beavers* con alfabetos no binarios,  $BB(n, m)$ , donde  $m$  es el tamaño del alfabeto; o la función de conteo de unos  $\Sigma(n, m)$ , que cuenta los unos escritos en la cinta en vez de los pasos dados.

## 7. CONCLUSIONES

Descubrir el valor de  $BB(5)$  ha sido una tarea titánica, pero es solo un paso más en el juego, porque el margen teórico antes de llegar al límite de ZFC aún es amplio, siempre que las conjeturas de Aaronson resulten equivocadas. Por ahora, los esfuerzos de la comunidad se centran en identificar las máquinas en  $\mathcal{M}_6^2$  y  $\mathcal{M}_3^3$  que muestren comportamientos más problemáticos, una especie de *Skelets* modernas, pero con nombres de críptidos, como *Hydra* [15], *Antihydra* [17] y *Bigfoot* [16]. Determinar la parada de estas máquinas parece demasiado difícil por ahora, pero nunca descartemos que un matemático aficionado con conocimientos de *Rocq* encuentre el método para lograrlo.

Desde luego, la capacidad de comunidades de aficionados para hacer descubrimientos matemáticos rigurosos ha quedado más que demostrada. Y aunque sea poco probable que en nuestras vidas conozcamos el valor de  $BB(6)$  o  $BB(3, 3)$ , su búsqueda no es en vano, pues nos ha proporcionado ya una nueva generación de matemáticos listos para colaborar en comunidad, personas que quizá nunca se habrían interesado por la disciplina si no fuese por esta curiosa función. Quién sabe si alguien, algún día, encontrará en este zoo de castores la solución a un gran problema abierto de las matemáticas. Solo queda esperar.

## 8. LECTURAS RECOMENDADAS

En el artículo de Aaronson que cité varias veces a lo largo del texto [1] se hace una muy buena introducción al tema. También recomiendo el *blog* de Shawn Ligocki, <https://www.sligocki.com>, donde se discuten máquinas particulares; la página de <https://BBChallenge.org>, que es muy completa; y la web de Heiner Marxen, <https://turbotm.de/~heiner/BB/>, que detalla la historia previa a la fundación de *BBChallenge*.

## REFERENCIAS

- [1] S. AARONSON, The Busy Beaver frontier, *ACM SIGACT News* **51** (2020), no.3, 32–54.
- [2] S. AARONSON, *Three announcements*, blog, may. 2016. <https://scottaaronson.blog/?p=2741> (visitado 25-08-2025).
- [3] S. AARONSON, *BusyBeaver(6) is really quite large*, blog, jun. 2025. <https://scottaaronson.blog/?p=8972> (visitado 25-08-2025).
- [4] S. AARONSON, *The 8000th Busy Beaver number eludes ZF set theory: new paper by Adam Yedidia and me*, blog, may. 2016. <https://scottaaronson.blog/?p=2725#comment-1091551> (visitado 07-04-2026).
- [5] A. H. BRADY, The determination of the value of Rado's noncomputable function  $\Sigma(k)$  for four-state Turing machines, *Math. Comp.* **40** (1983), no.162, 647–665.

- [6] D. BRIGGS, *Turing2010*, archive. oct. 2020. <https://github.com/danbriggs/Turing2010> (visitado 16-04-2025).
- [7] P. ERDŐS, Some unconventional problems in number theory, *Math. Mag.* **52** (2) (1979), 67–70.
- [8] H. FRIEDMAN, *IFOMI 273:Sigma01 optimal/size*, archive. mar. 2006. <https://web.archive.org/web/20240918084441/https://cs.nyu.edu/pipermail/fom/2006-March/010279.html> (visitado 16-04-2025).
- [9] G. GEORGIEV, *Busy Beaver nonregular machines for class  $TM(5)$* , blog, may. 2003. <https://skelet.ludost.net/bb/nreg.html> (visitado 21-04-2025).
- [10] C. GOLDBACH, *Correspondence of Leonhard Euler*, jun. 1742. <https://web.archive.org/web/20250120135755/http://eulerarchive.maa.org/correspondence/letters/000765> (visitado 17-02-2026).
- [11] K. GÖDEL, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I, *Monatsh. Math. Phys.* **38** (1931), no. 1, 173–198.
- [12] J. HERTEL, Computing the uncomputable Rado sigma function, *The Mathematical Journal* **11**(2) (2009).
- [13] S. LIGOCKI, *Skelet #1 is infinite ... we think*, blog, mar. 2023. <https://www.sligocki.com/2023/03/13/skelet-1-infinite.html> (visitado 16-04-2025).
- [14] S. LIGOCKI, *Skelet #10: Double Fibonacci counter*, blog, mar. 2023. <https://www.sligocki.com/2023/03/14/skelet-10.html> (visitado 16-04-2025).
- [15] S. LIGOCKI, *BB(2,5) is Hard (Hydra)*, blog, may. 2024. <https://www.sligocki.com/2024/05/10/bb-2-5-is-hard.html> (visitado 16-04-2025).
- [16] S. LIGOCKI, *BB(3,3) is hard (Bigfoot)*, blog, may. 2024. <https://www.sligocki.com/2023/10/16/bb-3-3-is-hard.html> (visitado 16-04-2025).
- [17] S. LIGOCKI, *BB(6) is hard (Antihydra)*, blog, jun. 2024. <https://www.sligocki.com/2024/07/06/bb-6-2-is-hard.html> (visitado 16-04-2025).
- [18] S. LIN, *Computer studies of Turing machine problems*, Tesis doctoral, Ohio State University, 1965. [https://etd.ohiolink.edu/acprod/odb\\_etd/etd/r/1501/10?clear=10&p10\\_accession\\_num=osu1486554418657614](https://etd.ohiolink.edu/acprod/odb_etd/etd/r/1501/10?clear=10&p10_accession_num=osu1486554418657614)
- [19] S. LIN Y T. RADO, Computer studies of Turing machine problems, *J. Assoc. Comput. Mach.* **12.2** (1965), 196–212.
- [20] H. MARXEN Y J. BUNTROCK, Attacking the Busy Beaver 5, *Bulletin of the EATCS* **40** (1990), 247–251.
- [21] T. RADO, On non-computable functions, *Bell System Tech. J.* **41** (1962), 877–884.
- [22] S. O'REAR, *metamath-turing-machines*, archive, may. 2016. <https://github.com/sorear/metamath-turing-machines> (visitado 03-05-2026).
- [23] J. RIEBEL, *The undecidability of BB(748): Understanding Gödel's incompleteness theorems*, Bachelor's thesis, Universität Augsburg, 2023. <https://www.ingo-blechschmidt.eu/assets/bachelor-thesis-undecidability-bb748.pdf>

- [24] B. RIEMANN, Ueber die Anzahl der Primzahlen unter einer gegebenen Grösse, *Monatsberichte der Königlich Preussischen Akademie der Wissenschaften zu Berlin* (1859). <https://www.emis.de/classics/Riemann/Zeta.pdf>
- [25] T. STÉRIN ET AL., *Coq-BB5*, archive, mar. 2025. <https://github.com/ccz181078/Coq-BB5> (visitado 16-04-2025).
- [26] A. M. TURING, On computable numbers, with an application to the Entscheidungsproblem, *Proc. London Math. Soc. (2)* **42** (1936), no.3, 230–265.
- [27] C. XU, *Skelet #17 and the fifth Busy Beaver number*, version number 2, jul. 2024. <https://arxiv.org/abs/2407.02426>
- [28] A. YEDIDIA Y S. AARONSON, A relatively small Turing machine whose behavior is independent of set theory, *Complex Systems* **25** (2016), no. 4, 297–327.

SERGIO MIGUÉNS IGLESIAS, AKAMAI TECHNOLOGIES

Correo electrónico: [yo@sergio.sh](mailto:yo@sergio.sh)

Página web: <https://sergio.sh>